

Architecture Logicielle

TP 1 : Application E-commerce Monolithique

Présenté par :
Haytham KENNOUZ

Encadré par :
Prof.ELBAGHAZAOU

2 mars 2026

Table des matières

1	Introduction	3
2	Objectifs du TP	3
3	Architecture Technique	3
3.1	Structure du Projet	3
4	Implémentation	3
4.1	Modèle de Données	3
4.2	Couche Service et Contrôleur	4
5	Résultats et Validation	4
6	Conclusion	6

1 Introduction

Dans le cadre du module d'Architecture Logicielle, ce premier travail pratique (TP) porte sur la conception et le développement d'une application monolithique. Une architecture monolithique est un modèle traditionnel de développement logiciel où toutes les fonctionnalités d'une application sont regroupées au sein d'une seule et même unité déployable.

Ce rapport détaille l'implémentation d'une plateforme e-commerce simplifiée permettant la gestion des produits et des catégories.

2 Objectifs du TP

Les principaux objectifs de ce TP étaient :

- Comprendre les principes fondamentaux de l'architecture monolithique.
- Maîtriser le framework **Spring Boot** pour le développement d'API REST.
- Mettre en œuvre le pattern **Repository** avec Spring Data JPA.
- Gérer la persistance des données avec une base de données relationnelle (**PostgreSQL**).
- Implémenter des relations entre entités (Many-to-One).

3 Architecture Technique

L'application repose sur une pile technologique moderne et robuste :

- **Langage** : Java 17
- **Framework** : Spring Boot 3.2.2
- **Gestionnaire de dépendances** : Maven
- **Persistance** : Spring Data JPA (Hibernate)
- **Base de données** : PostgreSQL
- **Utilitaire** : Lombok (pour la réduction du code boilerplate)

3.1 Structure du Projet

Le projet suit une organisation par packages fonctionnels (*package-by-feature*) :

- `com.ecommerce.monolith.product` : Gestion des produits (Modèle, Repository, Service, Controller).
- `com.ecommerce.monolith.category` : Gestion des catégories.

4 Implémentation

4.1 Modèle de Données

L'entité `Product` représente les produits de notre catalogue. Elle est liée à une `Category`.

```
1 @Entity
2 @Data
3 @Builder
4 @NoArgsConstructor
5 @AllArgsConstructor
```

```
6 public class Product {
7     @Id
8     @GeneratedValue(strategy = GenerationType.IDENTITY)
9     private Long id;
10
11     private String name;
12     private String description;
13     private Double price;
14     private Integer stock;
15
16     @ManyToOne
17     @JoinColumn(name = "category_id")
18     private Category category;
19 }
```

Listing 1 – Entité Product (extrait)

4.2 Couche Service et Contrôleur

La logique métier est encapsulée dans la couche Service, tandis que les points d'entrée API sont exposés via les contrôleurs REST.

TABLE 1 – Endpoints de l'API REST

Méthode	URL	Description
GET	/api/products	Lister tous les produits
POST	/api/products	Créer un nouveau produit
GET	/api/categories	Lister les catégories
GET	/api/products/search/by-name	Recherche par nom

5 Résultats et Validation

Le fonctionnement de l'application a été validé par une série de tests via Postman. Les captures suivantes confirment la bonne exécution des opérations CRUD.

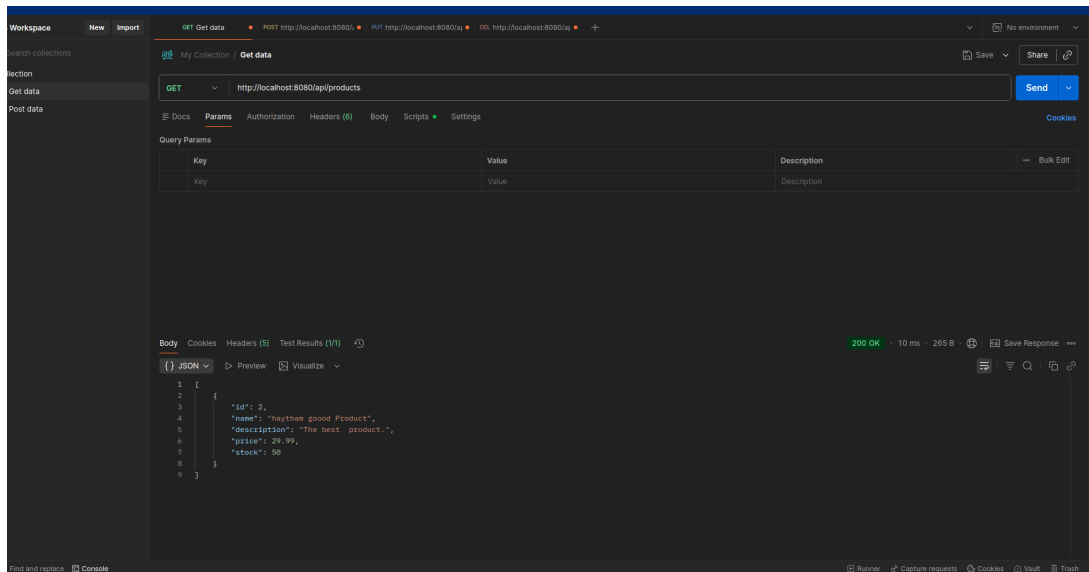


FIGURE 1 – Liste des produits via l'API REST

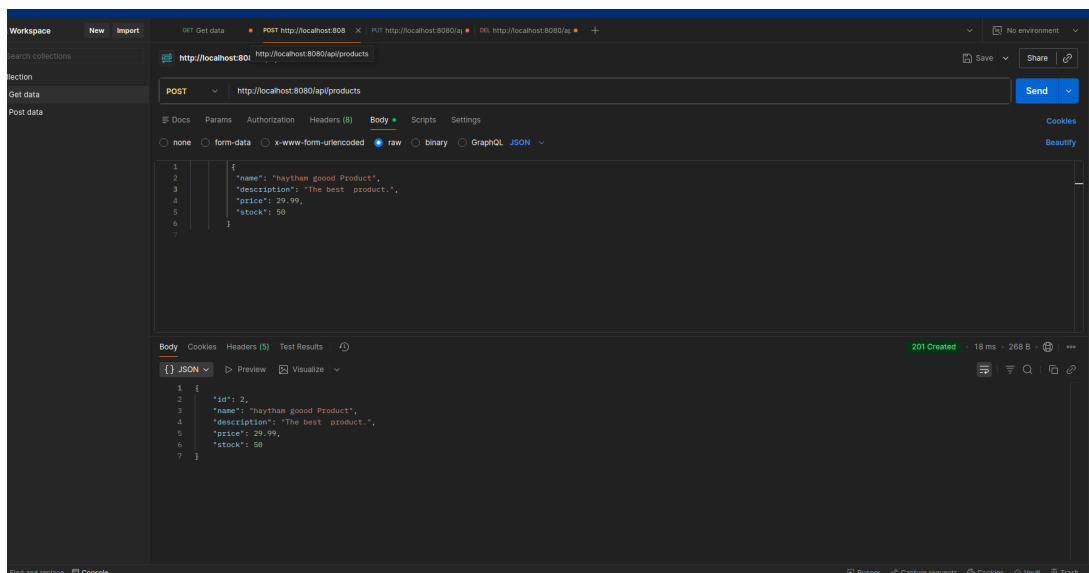


FIGURE 2 – Visualisation d'un produit spécifique

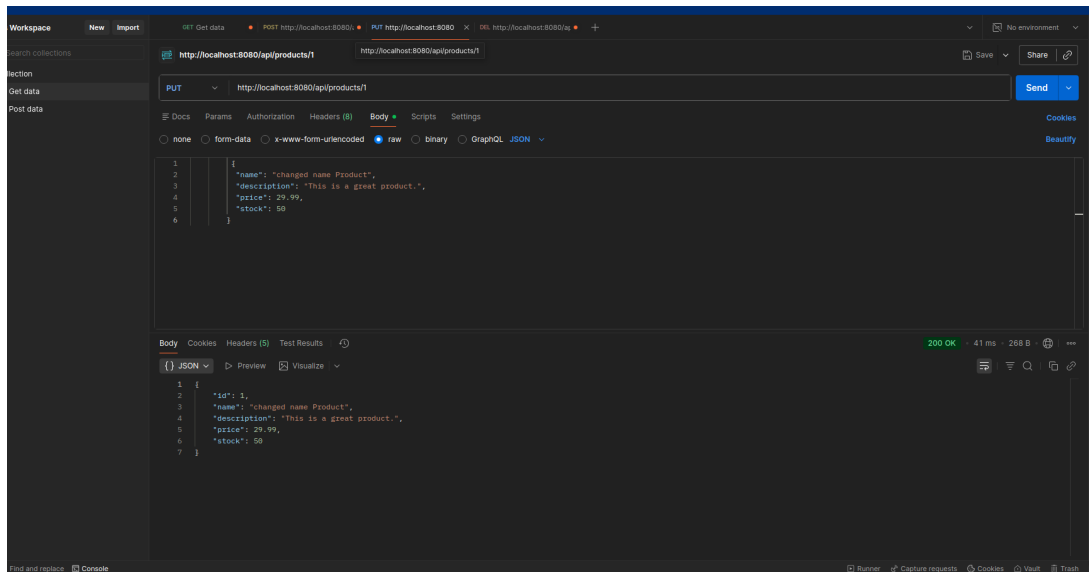


FIGURE 3 – Création d'une nouvelle catégorie

6 Conclusion

Ce TP a permis de concrétiser les concepts d'architecture monolithique. Bien que ce modèle présente des limites en termes de scalabilité, il reste une solution idéale pour les projets de petite à moyenne taille. L'utilisation de Spring Boot a grandement facilité le développement rapide de cette solution robuste.